

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - DESIGN TASK

COURSE NAME: INTERNET PROGRAMMING | COURSE CODE: CSA43

COURSE OUTCOME COVERED: CO-2 Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool: Design Task | Weightage: 20% | Total Marks: 20

STUDENT INFORMATION

Name:	N. Hima Harshitha
Register Number:	192371063
Course Name:	Internet Programming
Course Code:	CSA43
Assessment Tool:	Assessment Tool 2 – Design Task
Course Outcome:	CO2

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Identify key components required in a navigation bar for an e-commerce website.

A comprehensive navigation bar for an e-commerce platform serves as the primary visual portal and user routing control center. To maximize user conversion and seamless browsing, the following key components are required:

1. Brand Logo & Identity Header: Placed at the top-left section, serving as brand anchor and immediate direct hyperlink to the store's homepage.
2. Categorical Navigation Links: Primary links providing access to main store departments (e.g., Home, Shop, Categories, Deals, Contact).

3. Product Search Bar: A prominent input field with a search icon/button enabling users to query the inventory directly.
4. User Utility & Cart Module: Quick access icons for user account settings, wishlist, and shopping cart equipped with a dynamic item counter badge.
5. Responsive Hamburger Menu Button: A compact three-bar icon displayed on mobile screen sizes to open and close collapsible vertical navigation drawers.

(b) Design a responsive navigation bar using appropriate CSS techniques.

(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).

Short Explanation: The solution utilizes CSS Flexbox for horizontal alignment across desktop viewports. Media queries (@media (max-width: 768px)) dynamically transform the horizontal bar into a collapsible vertical menu on mobile screens. JavaScript handles the click event on the hamburger icon, toggling an active CSS class to reveal or hide navigation links smoothly.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Navigation Bar</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Times New Roman', serif; }
    .navbar { display: flex; justify-content: space-between; align-items: center; background: #1a252f; padding: 14px 28px; color: #fff; }
    .logo { font-size: 22px; font-weight: bold; color: #00bcd4; text-decoration: none; }
    .search-box { display: flex; flex: 0 1 280px; }
    .search-box input { width: 100%; padding: 7px; border: 1px solid #ccc; border-radius: 4px 0 0 4px; }
    .search-box button { padding: 7px 12px; background: #00bcd4; border: none; color: #fff; border-radius: 0 4px 4px 0; }
    .nav-links { display: flex; list-style: none; gap: 20px; align-items: center; }
    .nav-links a { color: #fff; text-decoration: none; }
    .cart-badge { background: #e74c3c; color: #fff; padding: 2px 6px; border-radius: 10px; font-size: 11px; }
    .hamburger { display: none; flex-direction: column; cursor: pointer; gap: 4px; }
    .hamburger span { width: 24px; height: 3px; background: #fff; border-radius: 2px; }
    @media (max-width: 768px) {
```

```

        .search-box { display: none; }
        .hamburger { display: flex; }
        .nav-links { position: absolute; top: 100%; left: 0; width: 100%; background:
#1a252f; flex-direction: column; padding: 18px 0; display: none; }
        .nav-links.active { display: flex; }
    }
</style>
</head>
<body>
    <nav class="navbar">
        <a href="#" class="logo">E-Store</a>
        <div class="search-box"><input type="text"
placeholder="Search..."><button>Search</button></div>
        <ul class="nav-links" id="navLinks">
            <li><a href="#">Home</a></li><li><a href="#">Shop</a></li><li><a
href="#">Categories</a></li><li><a href="#">Cart <span class="cart-
badge">3</span></a></li>
        </ul>
        <div class="hamburger"
onclick="toggleMenu()"><span></span><span></span><span></span></div>
    </nav>
    <script>
        function toggleMenu() {
document.getElementById('navLinks').classList.toggle('active'); }
    </script>
</body>
</html>

```

Browser Output Execution Screenshots:

Figure 1: Responsive Navigation Bar

Figure 2: Mobile Navigation

(d) Evaluate the usability and suggest improvements for better user experience.

Usability Evaluation: Flexbox maintains linear structure across desktop displays, while collapsing into a clean drawer on mobile devices to optimize space.

UX Improvements: Integrate ARIA attributes (aria-expanded), apply sticky positioning (position: sticky), and provide a mobile search overlay.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects.

Key properties include transform (translateY, scale), transition (duration, timing), box-shadow (depth simulation), opacity (overlay reveal), and filter (brightness/blur adjustments).

(b) Suggest appropriate CSS animation or transition for product interaction.

Using GPU-accelerated CSS transitions (transform: translateY(-8px) scale(1.02)) with a cubic-bezier timing function delivers smooth 60fps card hover effects without causing layout reflows.

(c) Design a hover effect for a product card.

Short Explanation: The product card elevates upward on hover with an expanded drop-shadow, while an 'Add to Cart' button slides up from the bottom container.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Product Card Hover Effect</title>
  <style>
    .product-card { width: 250px; background: #fff; border-radius: 8px; box-shadow: 0
4px 10px rgba(0,0,0,0.08); transition: transform 0.3s ease, box-shadow 0.3s ease;
cursor: pointer; }
    .product-card:hover { transform: translateY(-8px); box-shadow: 0 12px 22px
rgba(0,0,0,0.18); }
    .image-box { height: 180px; background: #e2e8f0; display: flex; justify-content:
center; align-items: center; position: relative; overflow: hidden; }
    .product-icon { font-size: 56px; transition: transform 0.4s ease; }
    .product-card:hover .product-icon { transform: scale(1.12); }
    .cart-btn { position: absolute; bottom: 10px; left: 50%; transform: translateX(-
50%) translateY(30px); background: #00bcd4; color: #fff; padding: 6px 14px; border:
none; border-radius: 15px; opacity: 0; transition: all 0.3s ease; }
    .product-card:hover .cart-btn { opacity: 1; transform: translateX(-50%)
translateY(0); }
  </style>
</head>
<body>
  <div class="product-card">
    <div class="image-box"><div class="product-icon">📺</div><button class="cart-
btn">Add to Cart</button></div>
    <div style="padding:15px;"><div style="font-size:11px;
color:#64748b;">Electronics</div><div style="font-size:16px; font-
weight:bold;">Wireless Headphones</div><div style="color:#e74c3c; font-
weight:bold;">$129.99</div></div>
  </div>
```

```
</body>
</html>
```

Figure 3: Product Hover Effect

(d) Evaluate performance and user experience considerations.

Performance: Hardware-accelerated transform properties prevent repaints. UX: Mobile touch fallbacks ensure 'Add to Cart' remains accessible without hover states.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

Requirements include a full-width header bar, vertical sidebar navigation column, responsive multi-column KPI card grid, and multi-row analytics panels.

(b) Compare Flexbox and Grid based on layout needs.

Criteria	CSS Flexbox	CSS Grid Layout
Dimension	One-Dimensional (Row or Col)	Two-Dimensional (Row and Col)
Design Approach	Content-driven	Layout-driven
Track Control	Requires width percentages	Native grid-template-columns
Dashboard Role	Micro-layouts (card contents)	Macro-layouts (overall scaffold)

(c) Choose the most suitable layout technique and justify your choice.

Choice: Hybrid Approach (CSS Grid for macro two-dimensional page structure + CSS Flexbox for micro internal component alignment).

(d) Evaluate the scalability and responsiveness of the chosen approach.

Grid auto-fit tracks enable scalable wrapping of KPI widgets without requiring repetitive media queries.

Figure 4: Dashboard Layout

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout.

Header & Navigation Banner, Main Article Container (Hero Image, Headline, Metadata, Body), Sidebar Panel (Author Info, Categories), Footer.

(b) Design a mobile-first layout using CSS.

(c) Demonstrate how the layout scales for larger screens using media queries.

Mobile-first base CSS styles narrow viewports without media queries. Progressive enhancement via @media (min-width: 1024px) expands the layout into a two-column grid.

Figure 5: Blog Layout

(d) Evaluate accessibility and readability of the design.

Maintains optimal line lengths (60-75 characters) and uses semantic HTML5 landmarks for assistive technology accessibility.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

Events: click (toggling dropdowns), mouseenter/mouseleave (hover handling), keydown (Escape key navigation), focusout/blur (focus shift closing).

(b) Design event handling logic for menu toggle functionality.

(c) Implement interaction for user actions (click/hover).

Handles both click and hover interaction cycles cleanly, including document outside clicks and Escape key listeners.

Figure 6: Dynamic Menu

(d) Evaluate performance and suggest improvements.

Optimizes event handler scoping and suggests dynamic ARIA state updates (aria-expanded).